Oliver Krauss

Stored Procedures, Functions and Triggers

RDB | MTD.ba VZ SS26

Hagenberg 15.04.2026

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

What are Stored Procedures?

- A stored procedure is a precompiled collection of one or more SQL statements stored under a name.
- Can be invoked using a single call statement.
- Helps in encapsulating complex business logic.
- Reduces network traffic and improves performance.
- Supports parameters (IN, OUT, INOUT).

Stored Procedure: Basic Structure

```
1   CREATE PROCEDURE ListQuests()
2   BEGIN
3     SELECT * FROM Quests;
4   END;
```

`CREATE PROCEDURE` defines a reusable named block of SQL

`BEGIN` marks the start of the procedure's logic

SQL logic (e.g. SELECT, INSERT, etc.) goes here

`END;` closes the procedure definition

Custom Delimiters in Stored Procedures

In some cases (MySQL CLI for example) the ";" needs to be overriden for safe procedure handling.

```
1  DELIMITER //
2
3  CREATE PROCEDURE ListQuests()
4  BEGIN
5    SELECT * FROM Quests;
6  END //
7
8  DELIMITER ;
```

Changes the command terminator to `//`

Allows procedure body to use `;` safely

Resets delimiter back to semicolon

Stored Procedure Parameters: Overview

Parameters allow stored procedures to interact dynamically with input and output values. There are three types:

- **IN**:
  - Default type.
  - Used to pass a value *into* the procedure.
  - Cannot be modified by the procedure.
- **OUT**:
  - Used to return a value *from* the procedure.
  - Must be assigned inside the procedure.
- **INOUT**:
  - Acts as both input and output.
  - Passed value can be read and overwritten by the procedure.

Stored Procedure: IN Parameter

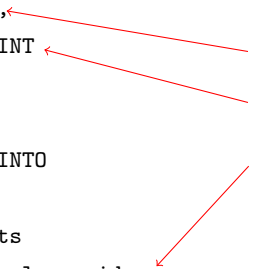
```
1  CREATE PROCEDURE GetPlayerByID(
2    IN p_id INT
3  )
4  BEGIN
5    SELECT * FROM Players
6    WHERE PlayerID = p_id;
7  END;
```

IN parameter to identify which player to retrieve

Returns the row for that specific PlayerID

Stored Procedure: OUT Parameter

```
1   CREATE PROCEDURE
        CountPlayerQuests(
2     IN player_id INT,
3     OUT quest_count INT
4   )
5   BEGIN
6     SELECT COUNT(*) INTO
        quest_count
7     FROM Player_Quests
8     WHERE PlayerID = player_id;
9   END;
```



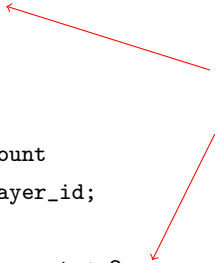
IN: Player ID as input

OUT: Result written to this variable

Query counts player quests and stores result

Stored Procedure: INOUT Parameter

```
1   CREATE PROCEDURE ApplyXPBoost(
2     IN player_id INT,
3     INOUT xp_amount INT
4   )
5   BEGIN
6     UPDATE Players
7     SET XP = XP + xp_amount
8     WHERE PlayerID = player_id;
9
10    SET xp_amount = xp_amount * 2;
11  END;
```



INOUT: Used as input and modified as output

Value is doubled after XP is applied

Calling Stored Procedures (SQL) I

To execute a stored procedure, use the `CALL` statement:

- **No parameters**:

```
CALL ListQuests();
```

- **IN parameter**:

```
CALL GetPlayerByID(5);
```

Calling Stored Procedures (SQL) II

- **IN + OUT parameter**:

```
SET @count := 0;
CALL CountPlayerQuests(5, @count);
SELECT @count;
```

- **INOUT parameter**:

```
SET @xp := 100;
CALL ApplyXPBoost(5, @xp);
SELECT @xp;
```

Stored Procedure in PHP

Using `PDO::query()` to access OUT parameters.

```php
$pdo = new PDO("mysql:host=localhost;dbname=game", "user", "pass");

$playerId = 5;
$pdo->query("SET @count = 0");
$pdo->query("CALL CountPlayerQuests($playerId, @count)");

$stmt = $pdo->query("SELECT @count AS quest_count");
$row = $stmt->fetch(PDO::FETCH_ASSOC);

echo "Player completed " . $row['quest_count'] . " quests.";
```

Fetching Results from Procedures

Procedures with SELECT statements return regular result sets.

```
$playerId = 5;
$result = $mysqli->query("CALL GetPlayerByID($playerId)");

while ($row = $result->fetch_assoc()) {
    echo "Name: " . $row['Name'] . "<br>";
    echo "Level: " . $row['Level'] . "<br>";
}
```

CALL in Prepared Statements

- **IN parameters**: Can be safely used with prepared statements.
- **OUT / INOUT parameters**: Cannot be bound directly — use `@var` session variables.
- **Security**: Always validate inputs when injecting into raw `CALL`.

Why OUT Parameters Can't Be Used in Prepared Statements

- **Prepared statements** can only bind variables to values passed into the SQL engine.
- `@session_variables` (like `@count`) are evaluated by the SQL engine itself.
- Binding a placeholder to an `@variable` is not supported.
- Therefore, OUT/INOUT parameters must be accessed using:
  - Regular (non-prepared) `CALL`
  - Separate `SELECT @var`

OUT Parameters in PHP with PDO

```php
// Step 1: Declare session variable
$pdo->query("SET @quest_count = 0");

// Step 2: Call the procedure
$pdo->query("CALL CountPlayerQuests($playerId, @quest_count)");

// Step 3: Retrieve OUT value
$stmt = $pdo->query("SELECT @quest_count AS quests");
$row = $stmt->fetch(PDO::FETCH_ASSOC);
```

Benefits of Stored Procedures

- **Modularity**: Write once and reuse many times.
- **Performance**: Fewer calls from application layer.
- **Security**: Permissions can be granted on procedures instead of tables.
- **Maintainability**: Centralized logic for complex operations.
- **Transaction Control**: Can include BEGIN, COMMIT, ROLLBACK inside.

Limitations of Stored Procedures

- **Portability**: Syntax differs between database systems (e.g. MySQL vs PostgreSQL vs SQL Server).
- **Debugging**: Harder to debug than application code.
- **Complexity**: Can become difficult to manage if logic is too extensive.
- **Version Control**: Procedures live in the DB, not in regular code repositories.
- **Overuse**: Excessive use may obscure logic better placed in application layer.

Best Practices for Stored Procedures

- **Name clearly**: Use prefixes like `sp_`, e.g. `sp_GetActivePlayers`.
- **Keep short and focused**: Each procedure should do one task well.
- **Avoid business logic**: Keep complex business rules in application code.
- **Use parameters**: Prefer `IN`, `OUT`, and `INOUT` for flexibility.
- **Document thoroughly**: Describe purpose, parameters, and side effects.

Stored Procedures: Summary

- Stored procedures are reusable, server-side logic blocks written in SQL.
- Useful for encapsulating repetitive logic like quest assignment, XP calculation, etc.
- They offer performance, modularity, and security benefits—but are not a silver bullet.
- Understand the trade-offs and follow best practices to avoid maintenance issues.
- Knowing when **not** to use stored procedures is as important as knowing how to use them.

What is a Stored Function?

- A stored function is a reusable SQL block that **returns a single value**.
- Can be used **in expressions**, **SELECT**, **WHERE**, **SET**, etc.
- Similar to stored procedures but must have:
  - `RETURNS` clause (data type)
  - `RETURN` statement inside the body
- Useful for **calculations**, **lookups**, or **derived data**.

Stored Function: Calculate XP Needed

Stored functions are ideal for encapsulating logic used in multiple places — especially inside queries.

```
1   CREATE FUNCTION
        XPRequiredForLevel(
2     level INT
3   )
4   RETURNS INT
5   BEGIN
6     DECLARE required_xp INT;
7     SET required_xp = level * 1000;
8     RETURN required_xp;
9   END;
```

Input parameter: level

Simple calculation (XP = level × 1000)

RETURN must be used to give back a value

Stored Procedure vs Stored Function

	Stored Procedure	**Stored Function**
Return type	None or OUT/INOUT	Must return a single value
Use in SQL	Can't be used in expressions	Can be used in SELECT, WHERE, etc.
Called with	`CALL`	Used like built-in function
Focus	Actions or side effects	Return value based on input

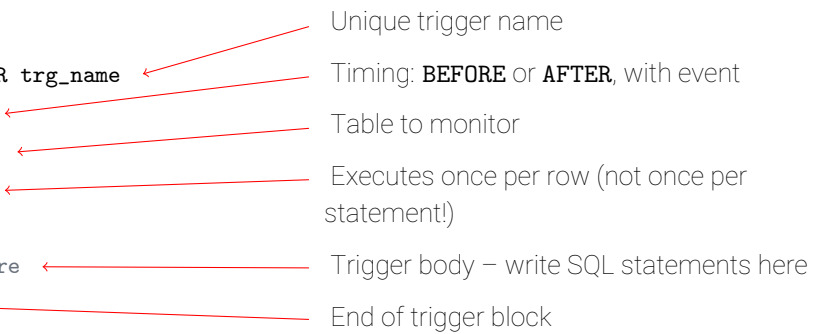
What is a Trigger?

A trigger is a piece of SQL logic that runs **automatically** in response to data changes.

- Attached to a specific table
- Executes **before** or **after** an `INSERT`, `UPDATE`, or `DELETE`
- Used to:
  - Enforce rules
  - Log changes
  - Maintain derived data
- Cannot return values like functions

Trigger Syntax Overview

```
1   CREATE TRIGGER trg_name
2   AFTER INSERT
3   ON table_name
4   FOR EACH ROW
5   BEGIN
6     -- logic here
7   END;
```



OLD vs NEW in Triggers

When writing a trigger, SQL gives you access to special row references:

- `NEW` – Refers to the *new version* of the row
  - Used in `INSERT` and `UPDATE` triggers
  - Can be read or modified in `BEFORE` triggers
- `OLD` – Refers to the *existing version* of the row
  - Used in `UPDATE` and `DELETE` triggers
  - `READ-ONLY` — can't be changed

Examples:

- `NEW.XP` The XP value being inserted or updated
- `OLD.Name` The original name before an update

OLD vs NEW: Reference Table

Trigger Type	OLD	NEW
`INSERT`	-	New row values
`UPDATE`	Previous values	Updated values
`DELETE`	Deleted row values	-

- `OLD` is always read-only
- `NEW` can be modified only in `BEFORE INSERT` or `BEFORE UPDATE`

Trigger: Log XP Changes Using OLD and NEW

```
1   CREATE TRIGGER LogXPChange
2   AFTER UPDATE ON Players
3   FOR EACH ROW
4   BEGIN
5     IF OLD.XP != NEW.XP THEN
6       INSERT INTO XP_Log
7       (PlayerID, OldXP, NewXP,
        ChangedAt)
8       VALUES
9       (OLD.PlayerID, OLD.XP, NEW.XP
        , NOW());
10    END IF;
11  END;
```

AFTER Trigger: Log New Player Registrations

AFTER INSERT runs after the new player is saved — great for logging or syncing.

```
1   CREATE TRIGGER LogNewPlayer
2   AFTER INSERT ON Players
3   FOR EACH ROW
4   BEGIN
5     INSERT INTO Player_Log
6     (PlayerID, Event, LogTime)
7     VALUES
8     (NEW.PlayerID, 'Registered',
        NOW());
9   END;
```

Triggered when a row is inserted into **Players**

NEW allows access to inserted row values

BEFORE Trigger: Prevent Negative XP

Use **BEFORE** triggers to validate or modify data before it hits the table.

```
1  CREATE TRIGGER PreventNegativeXP
2  BEFORE UPDATE ON Players
3  FOR EACH ROW
4  BEGIN
5    IF NEW.XP < 0 THEN
6      SET NEW.XP = 0;
7    END IF;
8  END;
```

BEFORE UPDATE lets us change the new value

Ensures XP doesn't drop below zero

Triggers: Where to use them

- **Logging**: Automatically log changes (e.g. XP gain/loss, quest completion)
- **Validation Rules**: Enforce domain rules (e.g. no negative XP)
- **Derived Data**: Maintain summary or stats tables in sync
- **Cleanup**: Cascade deletes or clean up related records
- **Monitoring**: Alert or log suspicious updates

Triggers: Limitations

- **Hard to Debug**: Trigger logic is invisible during normal query flow
- **No Parameters**: Triggers can't take arguments like procedures or functions
- **No Views**: Triggers can't be defined on views
- **One per Event**: Only one `BEFORE` and one `AFTER` trigger allowed per table/event
- **Hidden Side Effects**: Makes behavior implicit — risk of confusion or recursion
- **Cascades**: Triggers can trigger other triggers - risk of endless loop and performance drops
- **Limited Portability**: Syntax and trigger features vary by DBMS

Triggers: Best Practices

- **Name clearly**: Use consistent naming like `trg_Table_Event`
- **Keep small and focused**: Avoid complex or multi-purpose trigger logic
- **Document effects**: Explain what the trigger does — it's not always obvious
- **Avoid recursion or chaining**: Don't let triggers trigger other triggers
- **Use for integrity or audit logic**: Keep business rules in app layer where possible

Trigger Summary

- Triggers execute **automatically** when data is inserted, updated, or deleted
- `BEFORE` triggers can validate or modify changes
- `AFTER` triggers are used for logging and side effects
- `OLD` and `NEW` give access to row values
- Great for **enforcing rules**, **logging actions**, and maintaining **data consistency**

Reminder: Keep logic readable and traceable — triggers are powerful but easy to misuse!